

You Always Getting Full: An AI Approach to Filler

Will Bradley (wjb247), Chandler Reyes (cr595), Rithya Sriram (ra596)

Keywords: game playing, minimax, reinforcement learning

Application setting: The iOS GamePigeon game “Filler”

We seek to compare different approaches to AI for games. Specifically, we will create several different types of artificial intelligence to play the abstract strategy game Filler. Filler is a simple, turn-based game for two players set on a fixed board of squares, each of which is one of a fixed number of colors. Either player is assigned one of two opposite corners of the board; every turn (ply), a player selects one of the available colors (all colors except the two most recently chosen by either player). That color is then applied all at once to all the squares that are reachable from the player’s corner while traveling through squares only of the corner’s color (at the start of the turn). Then the other player takes their turn. The game ends when one player controls at least half the squares; whoever has more is the winner (a tie is also possible).

By having our different algorithms and models compete against each other, we hope to investigate how different approaches to decision-making affect both game performance and efficiency of an AI player. We will use the classical minimax algorithm in a simple implementation and several optimized variants, including more advanced techniques such as alpha-beta pruning, optimized move ordering, iterative deepening, and better hand-crafted heuristics.

Additionally, we want to explore whether unsupervised machine learning can improve upon or complement our minimax approach. In particular, we may use reinforcement learning to develop an agent through repeated games or train a neural network to score moves within minimax. This will allow us to compare traditional, search-based approaches with learned strategies for playing Filler.

To evaluate the various algorithms, we will simulate thousands of random games between each pair of the algorithms— for example, games where player 1 uses the basic minimax implementation and player 2 uses a reinforcement learning model. We will also include some AIs that make random moves or make the worst move (according to some other AI). Then we can compile the head-to-head win-rates in a table. This lets us test whether a change to the architecture actually improves game performance.

We will also use a [flamegraph program](#) to easily find CPU bottlenecks. If two models win at roughly equal rates but one takes much longer or uses a lot more memory than the other, we will take that into consideration. We may interact with the system for debugging purposes, but it’s not a focus of our project to have our AIs play against people.

We have drafted a complete project timeline from now until the end of the semester:

- 9/12-9/25 (2 weeks): Establish library and abstract interface so we can benchmark different implementations easily. Add the implementation of the greedy algorithm, compare it to making random moves. Implement core minimax algorithm.
- 9/26-10/16 (3 weeks): Create variations of minimax with alpha-beta pruning, better heuristics for the move ordering, and possibly iterative deepening, as well as any other advanced techniques we find in our research. In parallel, start work on the RL/other machine learning approaches to see if machine learning can come up with better search heuristics than we can. In a similar vein, we may also experiment with [training a neural network to evaluate moves](#)— the Stockfish algorithm for chess does something like this.
- 10/17-10/23 (1 week): Prepare progress report. To the extent possible, keep working on the software as well. Additional time for set-backs.
- 10/24-11/20 (4 weeks): Finish working on models. In parallel with this, and also after, evaluate the various models by win-rate and resource use.
- 11/21-12/4 (2 weeks): Most of the writing will happen here, so we should have the evaluation done by this time.
- 12/5-end (1 week): Finish writing project presentation, the group report, and individual project reports. Practice and deliver final presentation.

Our project does not depend on existing data, since we will (uniformly) randomly generate game instances.

As for software, we will use Java or OCaml for the solver library, LaTeX for writing the report, and Python for any graphs. We will also measure performance in the solver library with a profiler and/or flamegraph program.

This project is not connected to any outside work any of us are doing.

References:

- Wikipedia pages for minimax, iterative deepening, combinatorial optimization, etc.
- https://en.wikipedia.org/wiki/Evaluation_function#Neural_networks
- <https://blogs.cornell.edu/info2040/2022/09/30/game-theory-how-stockfish-mastered-chess/>
- [The "Making Stockfish Stronger" series on Youtube](#)